

Guia HPC Heisenberg

Fundamentos e Prática

Mateus de Jesus Mendes
`mateus25032@ilum.cnpem.br`

Conteúdo

1	Introdução ao HPC	3
1.1	O que é HPC?	3
1.2	Motivação	3
1.3	Arquitetura de um <i>cluster</i> HPC	3
1.4	CPU vs. GPU	5
1.5	Conceitos Fundamentais	5
1.6	O <i>cluster</i> Heisenberg	6
2	Acesso e Ambiente de Trabalho	6
2.1	Formas de Acesso	6
2.2	Acesso via Terminal	7
2.3	Acesso via WinSCP	7
2.4	Acesso via VS Code (Remote-SSH)	7
2.5	Acesso via VPN	8
3	Comandos Bash Essenciais	8
3.1	Navegação e Inspeção	8
3.2	Manipulação de Arquivos e Diretórios	8
3.3	Visualização de Conteúdo	9
3.4	Monitoramento de Recursos	9
3.5	Comandos do <i>cluster</i> (SLURM)	9
3.6	Gerenciamento de Módulos	10
3.7	Download de Arquivos e Repositórios	10
4	Ambientes Virtuais e Dependências	10
4.1	Por que Usar Ambientes Virtuais?	10
4.2	Conda: Gerenciamento de Ambientes	11
4.3	Principais Problemas com Dependências	13
5	Modos de Execução	13
6	Gerenciador de Jobs: SLURM	14
6.1	O que é o SLURM?	14
6.2	Estrutura Geral de um Script SLURM	14
6.2.1	Script SLURM para Jupyter Notebook	14
6.2.2	Script SLURM para Script Python	15
6.3	Parâmetros Principais do SLURM	16

7	Submissão, Monitoramento e Diagnóstico	16
7.1	Submetendo e Cancelando Jobs	16
7.2	Monitorando a Execução	17
8	Workflow Completo	17
9	Boas Práticas	18
9.1	Uso Responsável do <i>cluster</i>	18
9.2	Organização de Experimentos	18
9.3	Boas Práticas de Submissão	18
9.4	Logging e Registro de Execução	19
9.5	Reprodutibilidade	20
10	Erros Comuns e Debugging	21
10.1	Job Preso em PENDING	21
10.2	Falta de Memória (<i>Out of Memory</i>)	21
10.3	Uso Incorreto de GPU	21
10.4	Kernel Morto no Jupyter / Dependências Quebradas	22
A	Tutorial de Acesso à VPN	22
B	Referência Rápida de Comandos SLURM	22
C	Referência Rápida de Comandos Conda	23

Observação: As seções e subseções marcadas em **vermelho** são destinadas à consulta para aqueles que desejam efetuar uma breve revisão de acesso rápido ao *cluster* Heisenberg.

1 Introdução ao HPC

1.1 O que é HPC?

High Performance Computing (HPC)

Conjunto de técnicas, arquiteturas e ferramentas que permitem executar computações de grande escala de forma eficiente, explorando paralelismo e recursos distribuídos.

Em sua essência, o HPC existe para superar as limitações de uma única máquina. Um computador pessoal moderno pode ter, tipicamente, 8 a 16 núcleos de processamento e alguns gigabytes de RAM. Embora suficiente para tarefas cotidianas, esse hardware não é capaz de treinar modelos de *Deep Learning* em milhões de exemplos, simular sistemas moleculares por nanosegundos ou processar imagens científicas de alta resolução em tempo hábil.

A solução é conectar dezenas ou centenas de máquinas em uma rede de alta velocidade e coordenar sua execução, sendo esse o princípio fundamental de um *cluster* HPC.

1.2 Motivação

Por que um pesquisador ou estudante deve aprender a usar HPC? Existem pelo menos quatro razões centrais:

- **Escala de dados:** experimentos científicos modernos geram *datasets* que facilmente excedem a capacidade de RAM de um único computador. No contexto do CNPEM, por exemplo, imagens de tomografia síncrotron ou espectroscópios de alta resolução produzem terabytes de dados por campanha experimental.
- **Demanda computacional:** modelos de aprendizado de máquina de grande porte, como *Transformers* e redes convolucionais profundas, exigem GPUs de alta capacidade — e, frequentemente, múltiplas delas operando em paralelo.
- **Escala de experimentos:** é comum querer treinar um mesmo modelo com dezenas de combinações de hiperparâmetros. Cada rodada pode demorar horas em uma máquina comum; em um *cluster*, todas rodam simultaneamente.
- **Reprodutibilidade:** ao submeter um job para o *cluster*, o ambiente de execução fica registrado e pode ser recriado com precisão, facilitando a sua reprodução por terceiros.

1.3 Arquitetura de um *cluster* HPC

Um *cluster* é composto por um conjunto de máquinas, denominadas **nós**, interligadas por uma rede de alta velocidade. Do ponto de vista do usuário, é como se houvesse

uma única máquina muito poderosa, contudo, internamente, o trabalho é distribuído entre múltiplas unidades independentes.

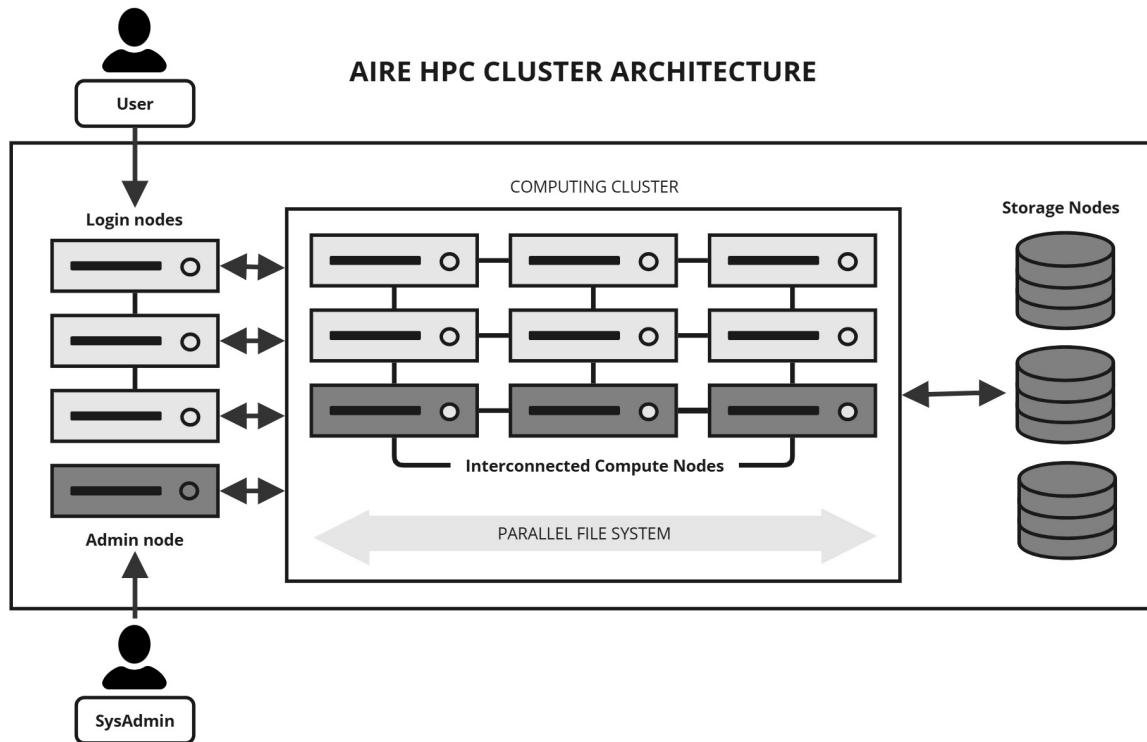


Figura 1: Exemplo da arquitetura de um *cluster* HPC. Fonte: https://arcdocs.leeds.ac.uk/aire/system/hpc_architecture.html

Os componentes fundamentais são:

- **Login Node** (*nó de acesso*): ponto de entrada do usuário no *cluster*. É por aqui que se conecta via SSH, edita *scripts*, transfere arquivos e submete jobs. **Nunca execute computações pesadas no login node** — isso prejudica todos os usuários do sistema.
- **Compute Nodes** (*nós de processamento*): máquinas dedicadas à execução de jobs. São gerenciados pelo *scheduler* (no nosso caso, o SLURM), que aloca recursos automaticamente conforme a demanda e a disponibilidade.
- **Storage** (*armazenamento compartilhado*): sistema de arquivos acessível por todos os nós simultaneamente. Tecnologias comuns incluem NFS (*Network File System*) e Lustre, otimizado para operações paralelas.
- **Network** (*rede de interconexão*): a velocidade da rede interna é crítica para o desempenho. Tecnologias como InfiniBand oferecem latência extremamente baixa e largura de banda da ordem de dezenas de Gb/s.

Observação

O *login node* é uma área de trabalho, não de processamento. Use-o apenas para navegar no sistema de arquivos, editar *scripts* e submeter jobs via **sbatch**. Computações pesadas executadas no *login node* afetam diretamente todos os outros usuários conectados.

1.4 CPU vs. GPU

A diferença entre CPU e GPU vai além do número de núcleos, definindo-se enquanto uma diferença filosófica no design:

Característica	CPU	GPU
Número de núcleos	8 – 128	Milhares (CUDA cores)
Frequência de <i>clock</i>	Alta (3–5 GHz)	Menor (1–2 GHz)
Arquitetura	Poucos núcleos potentes	Muitos núcleos simples
Ponto forte	Tarefas sequenciais complexas	Paralelismo massivo
Casos de uso	I/O, lógica de controle	Álgebra linear, ML
Memória	RAM compartilhada	VRAM dedicada (GDDR)

A GPU sobressai em operações que se repetem sobre grandes volumes de dados, como a multiplicação de matrizes que ocorre em cada camada de uma rede neural. Uma A100 da NVIDIA, por exemplo, realiza operações de ponto flutuante de 16 bits a mais de 300 TFLOPS (isso representa 300 trilhões de operações de ponto flutuante por segundo, um desempenho muito superior a qualquer CPU).

1.5 Conceitos Fundamentais

Antes de trabalhar com um *cluster*, é importante compreender alguns termos:

- **Processo:** instância de um programa em execução, com espaço de memória próprio e isolado. Dois processos não compartilham memória diretamente — a comunicação ocorre por mecanismos como *pipes*, *sockets* ou MPI.
- **Thread:** unidade de execução dentro de um processo. Múltiplas *threads* de um mesmo processo compartilham a memória, o que permite comunicação eficiente mas também introduz desafios de concorrência.
- **Kernel:** núcleo do sistema operacional; responsável por gerenciar recursos de hardware (CPU, memória, I/O) e mediar o acesso dos processos a esses recursos.
- **Job:** unidade de trabalho submetida ao *scheduler*. Um job descreve quantos recursos são necessários e quais comandos devem ser executados. Pode envolver um único processo ou múltiplos processos distribuídos em vários nós.

1.6 O *cluster* Heisenberg

O *cluster* Heisenberg, disponível aos alunos da Ilum, possui a seguinte configuração de nós de processamento:

Tabela 1: Configuração dos nós do *cluster* Heisenberg

Nó	CPUs	Threads	RAM (GiB)	GPUs
work0	128	256	503	—
work1	128	256	515	3× Tesla T4
work2	64	64	251	—
work3	64	64	251	—
work4	16	16	124	1× RTX 4090
work5	8	16	125	2× RTX 4090
work6	8	16	125	2× RTX 4090
work7	16	32	187	—
work8	48	96	157	1× RTX 3060

A escolha do nó ideal depende do tipo de tarefa: jobs que demandam muita memória RAM devem ser direcionados a `work0`, `work1` ou `work2/3`; jobs que envolvem múltiplos cálculos matriciais devem solicitar GPUs e serão alocados automaticamente em `work1`, `work4`, `work5` ou `work6`.

2 Acesso e Ambiente de Trabalho

2.1 Formas de Acesso

O Heisenberg pode ser acessado de três formas principais:

1. **Terminal / PowerShell:** acesso via linha de comando, ideal para operações automatizadas e integração com *scripts*.
2. **WinSCP:** cliente gráfico para transferência de arquivos via SFTP. Útil para fazer upload/download de diretórios e arquivos.
3. **VS Code com Remote-SSH:** integração completa do editor com o *cluster*. Permite editar arquivos, usar o terminal integrado e desenvolver de forma praticamente idêntica ao desenvolvimento local.

Recomendação

Para o fluxo de desenvolvimento habitual, recomenda-se o uso do **VS Code + Remote-SSH**. Ele oferece realce de sintaxe, *autocomplete*, depuração e terminal integrado — tudo dentro do ambiente do *cluster*.

2.2 Acesso via Terminal

No Windows, abra o PowerShell e para Linux/macOS, use o Terminal nativo, executando então o seguinte comando:

```
ssh usuarioXXX@heisenberg.cnpem.br
```

Após inserir a senha, você estará no *login node* do *cluster*.

Observação: Primeiro Acesso

Para o seu primeiro acesso ao HPC Heisenberg, a senha padrão será **temp123!@**. Essa senha deve ser alterada para uma outra senha forte, executando-se o comando **passwd**.

2.3 Acesso via WinSCP

1. Abra o WinSCP e clique em **Novo Site**
2. Configure os seguintes campos:
 - Protocolo de arquivo: **SFTP**
 - Host: **heisenberg.cnpem.br**
 - Porta: **22**
 - Usuário: **usuarioXXXX**
 - Senha: (mesma usada para o SSH)
3. Salve o perfil e clique em **Login**

2.4 Acesso via VS Code (Remote-SSH)

1. Instale a extensão Remote - SSH (Microsoft) no VS Code
2. Pressione a tecla F1 e selecione **Remote-SSH: Connect to Host**
3. Selecione a opção **"Add New SSH Host"**
4. Informe o seguinte comando: **ssh usuarioXXX@heisenberg.cnpem.br**
5. Selecione a opção **C:\Users\usuarioXXX\.ssh\config**
6. Pressione a tecla F1 e selecione a opção **"Connect in Current Window"**
7. Informe o sistema operacional do HPC (Linux)
8. Insira sua senha de acesso ao HPC
9. Digite **usuarioXXX@heisenberg.cnpem.br**
10. Selecione a opção **"Open folder"** e então selecione a pasta **"work"**
11. Insira a senha quando solicitado

2.5 Acesso via VPN

Nota

O *cluster* Heisenberg pode ser remotamente acessado, desde que a sua máquina local esteja conectada à VPN (FortiClient) do CNPEM.

3 Comandos Bash Essenciais

Dominar os comandos básicos do terminal é o primeiro passo para trabalhar com eficiência em um *cluster*. Esta seção cobre os comandos cotidianamente mais utilizados.

3.1 Navegação e Inspeção

```
ls                # lista arquivos e diretorios do diretorio atual
ls -lh           # listagem detalhada com tamanhos legiveis (KB, MB
                  ...)
ls -a            # inclui arquivos ocultos (iniciados com .)

pwd              # exibe o caminho do diretorio atual (Print Working
                  Directory)
cd /caminho     # navega para o caminho especificado
cd ..           # sobe um nivel na hierarquia de diretorios
cd ~            # vai para o diretorio home do usuario
```

3.2 Manipulação de Arquivos e Diretórios

```
mkdir nome       # cria diretorio
mkdir -p a/b/c   # cria diretorios aninhados (sem erro se ja
                  existirem)

cp src dst       # copia arquivo de src para dst
cp -r pasta/ dst/ # copia diretorio recursivamente

mv src dst       # move ou renomeia arquivo/diretorio

rm arquivo       # remove arquivo
rm -r pasta/     # remove diretorio e todo o seu conteudo
```

Cuidado com rm

Os comandos `rm -r` e `rm -rf` removem recursivamente todo o conteúdo de um diretório **sem mover para lixeira**. A remoção é imediata e irreversível. Use `rm -ri` para confirmação interativa antes de deletar cada item. Em sistemas compartilhados, nunca execute `rm -rf /` ou variantes que afetem o sistema de arquivos global.

3.3 Visualização de Conteúdo

```
cat arquivo.txt           # exibe o conteudo completo do arquivo
less arquivo.txt          # navegacao interativa (use q para sair
                           )
head -n 20 arquivo.txt    # exibe as primeiras 20 linhas
tail -n 20 arquivo.txt    # exibe as ultimas 20 linhas
tail -f arquivo.log       # acompanha o crescimento do arquivo em
                           tempo real
```

O comando `tail -f` é especialmente útil para monitorar logs de jobs enquanto eles executam, permitindo a visualização das novas linhas sendo adicionadas conforme o *script* imprime saída.

3.4 Monitoramento de Recursos

```
lscpu                     # exibe arquitetura da CPU e numero de nucleos
free -h                   # uso atual de RAM (h = formato legível por
                           humano)
du -sh *                  # tamanho de cada subdiretorio no local atual

nvidia-smi                # status das GPUs: uso, temperatura, processos
nvidia-smi -l 1           # atualiza a cada 1 segundo (Ctrl+C para parar)

htop                      # visualizacao dinamica de processos e uso de
                           CPU/RAM
top                        # versao mais simples, disponivel em qualquer
                           sistema
```

3.5 Comandos do *cluster* (SLURM)

```
sinfo                     # estado geral do cluster: nos e
                           particoes
```

```
squeue                                # lista todos os jobs em execucao/
    fila
squeue -u $USER                       # filtra apenas os jobs do usuario
    atual
scontrol show job <jobid>            # detalha a configuracao de um job
    especifico
```

3.6 Gerenciamento de Módulos

O *cluster* usa um sistema de módulos para disponibilizar versões específicas de softwares (CUDA, MPI, compiladores) sem conflito:

```
module avail                          # lista todos os modulos disponiveis
module list                          # lista os modulos atualmente carregados
module load cuda/12.2                # carrega a versao 12.2 do modulo CUDA
nvcc --version                       # verifica a versao do compilador CUDA
    carregado
```

3.7 Download de Arquivos e Repositórios

```
wget <url>                           # baixa um arquivo da internet
wget -c <url>                         # retoma um download interrompido
wget -O nome <url>                   # salva com nome especifico
wget -i lista.txt                   # baixa todos os URLs listados em um
    arquivo

curl -L -O <url>                     # alternativa ao wget, seguindo
    redirecionamentos

git clone <url>                      # clona repositorio Git (dataset,
    codigo, etc.)
```

4 Ambientes Virtuais e Dependências

4.1 Por que Usar Ambientes Virtuais?

Ambiente Virtual

Um ambiente virtual é um espaço isolado dentro do sistema operacional que permite gerenciar dependências, bibliotecas e versões de software de forma independente do restante do sistema.

No contexto de um *cluster* compartilhado, os ambientes virtuais são ainda mais importantes do que em máquinas pessoais, pelas seguintes razões:

- **Isolamento:** qualquer pacote pode ser instalado sem precisar de permissão de administrador.
- **Conflito de versões:** projetos diferentes frequentemente exigem versões incompatíveis de uma mesma biblioteca. Com ambientes separados, cada projeto vive em sua própria “bolha”.
- **Reprodutibilidade:** ao exportar o ambiente para um arquivo `.yaml` ou mesmo `requirements.txt`, qualquer pessoa (ou qualquer nó do *cluster*) pode recriar exatamente as mesmas condições.
- **Portabilidade:** o ambiente pode ser replicado entre diferentes máquinas do *cluster*, garantindo comportamento consistente.

4.2 Conda: Gerenciamento de Ambientes

Conda

Sistema de gerenciamento de pacotes e ambientes que permite instalar, atualizar e isolar dependências de software de forma controlada e reproduzível, independentemente do sistema operacional.

Verificando ambientes existentes:

```
conda env list    # lista ambientes; o ativo eh marcado com (*)
conda list        # lista todos os pacotes instalados no
                  ambiente atual
```

Criando um novo ambiente:

```
# Com a versao padrao de Python
conda create -n nome_env

# Especificando a versao do Python
conda create -n nome_env python=3.11

# Criando ja com pacotes instalados
conda create -n nome_env python=3.11 numpy pandas matplotlib

# A partir de um arquivo de configuracao (reprodutibilidade)
conda env create -f environment.yml
```

Ativando, desativando e removendo:

```
conda activate nome_env    # ativa o ambiente
```

```
conda deactivate          # desativa e volta ao base
conda env remove -n nome_env  # remove o ambiente
```

Instalando pacotes com Conda:

```
# Instalacao simples
conda install numpy

# Instalando multiplos pacotes
conda install numpy scipy pandas

# Instalando versao especifica
conda install numpy=1.26

# Instalando a partir de canais (muito importante em HPC)
conda install -c conda-forge scikit-learn

# Atualizar pacotes
conda update numpy

# Atualizar todo o ambiente
conda update --all

# Remove um pacote
conda remove numpy
```

Instalando pacotes com pip dentro do ambiente:

```
# Sempre com o ambiente ativado
conda activate nome_env

# Instalacao via pip
pip install nome_pacote

# Versao especifica
pip install nome_pacote==1.2.3
```

Exportando ambiente (reprodutibilidade):

```
conda env export > ambiente.yml
```

Ambiente no Script SLURM

O ambiente Conda **precisa ser explicitamente ativado dentro do script SLURM**. Caso contrário, o job rodará com o Python do sistema, sem os pacotes do seu ambiente. Sempre inclua `source ~/.bashrc` seguido de `conda activate nome_env` no início de todo script de submissão.

4.3 Principais Problemas com Dependências

- **Conflito de versões:** dois pacotes exigem versões incompatíveis de uma mesma dependência. Solução: criar ambientes separados por projeto.
- **Pacote compilado para outra arquitetura:** binários gerados em uma máquina local podem ser incompatíveis com a CPU/GPU do *cluster*. Solução: instalar e compilar diretamente no HPC.
- **Ambiente não encontrado no job:** o script SLURM não ativou o Conda corretamente. Solução: incluir `source ~/.bashrc` e `conda activate` no topo do script.

5 Modos de Execução

Há duas formas principais de executar código no *cluster*: via **Jupyter Notebook** ou via **scripts Python**. Cada abordagem tem suas vantagens e desvantagens.

Critério	Jupyter Notebook	Script Python
Interface	Gráfica e interativa	Linha de comando
Sessão SSH	Necessária	Não necessária
Integração SLURM	Limitada	Nativa
Jobs longos	Instável	Recomendado
Debugging	Fácil (células)	Por logs
Automação	Limitada	Total

Convertendo Notebooks para Scripts

Para converter um Jupyter Notebook em *script* Python no Jupyter Lab, basta selecionar no painel principal a seção **File**, então a opção **Save and Export Notebook As** e clicar em **Executable Script**.

O VS Code também permite converter um arquivo `.ipynb` para `.py` com um clique. Basta selecionar o Jupyter Notebook com o botão direito, e então clicar na opção **Import Notebook to Script**. Após isso, nomeie o arquivo gerado e salve-o (**Observação:** Não há a possibilidade de conversão pelo VSCode dentro do ambiente da extensão Remote-SSH).

6 Gerenciador de Jobs: SLURM

6.1 O que é o SLURM?

Script SLURM

Um script SLURM (*Simple Linux Utility for Resource Management*) é um arquivo de texto que descreve, de forma declarativa, os recursos computacionais necessários e os comandos a serem executados, permitindo que o *scheduler* aloque e execute o job no *cluster* de forma automática e eficiente.

O SLURM é o *scheduler* utilizado pelo Heisenberg. Sua função é receber pedidos de jobs dos usuários, colocá-los em fila, e aloca-los nos nós disponíveis conforme os recursos solicitados (CPUs, GPUs, memória, tempo). Quando um nó fica disponível, o SLURM automaticamente inicia a execução do próximo job na fila.

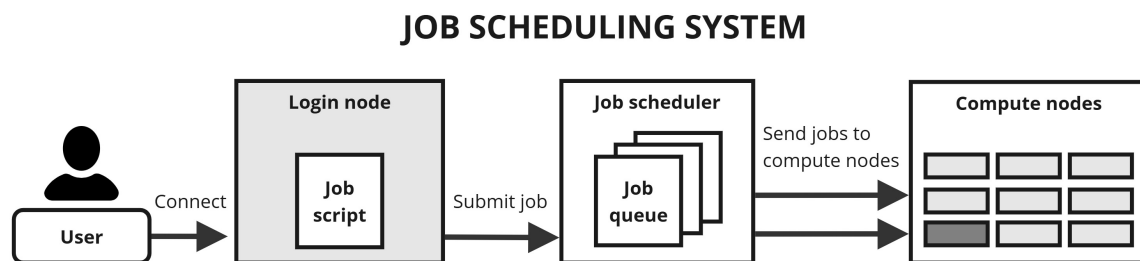


Figura 2: Representação do sistema de *scheduling* de um *cluster* HPC. Fonte: https://arcdocs.leeds.ac.uk/aire/system/hpc_architecture.html

6.2 Estrutura Geral de um Script SLURM

Um script SLURM é, essencialmente, um script Bash com diretivas especiais na forma `#SBATCH`. Essas diretivas são interpretadas pelo SLURM no momento da submissão e definem os recursos que o job irá utilizar.

6.2.1 Script SLURM para Jupyter Notebook

```

#!/bin/bash
#SBATCH --job-name=nome_job
#SBATCH --ntasks=1
#SBATCH --ntasks-per-node=1
#SBATCH --partition=gpu
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=1
#SBATCH --mem=16G
#SBATCH --odelist=work1

```

```

#SBATCH --time=DD-HH:MM:SS
#SBATCH --output=logs/saida_%j.out
#SBATCH --error=logs/erro_%j.err
#SBATCH --mail-type=BEGIN,END,FAIL
#SBATCH --mail-user=usuarioXXX@heisenberg.cnpem.br

# ---- Preparacao do ambiente ----
mkdir -p logs
source ~/.bashrc
conda activate meu_env

# ---- Execucao ----
ipaddress=$(ip addr | grep 172 | awk 'NR==1{print $2}' | sed 's
    !/23!!g' | sed 's!/0!!g')
echo $ipaddress

jupyter-notebook --ip=$ipaddress

```

O arquivo Shell pode ser obtido no repositório da monitoria, em: `job_ipynb.sh`

6.2.2 Script SLURM para Script Python

```

#!/bin/bash
#SBATCH --job-name=nome_job
#SBATCH --ntasks=1
#SBATCH --ntasks-per-node=1
#SBATCH --partition=gpu
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=1
#SBATCH --mem=16G
#SBATCH --odelist=work1
#SBATCH --time=DD-HH:MM:SS
#SBATCH --output=logs/saida_%j.out
#SBATCH --error=logs/erro_%j.err
#SBATCH --mail-type=BEGIN,END,FAIL
#SBATCH --mail-user=usuarioXXX@heisenberg.cnpem.br

# ---- Preparacao do ambiente ----
mkdir -p logs
source ~/.bashrc
conda activate meu_env

```



```
# ---- Execução ----
python script.py --argumentos
```

O arquivo Shell pode ser obtido no repositório da monitoria, em: `job_py.sh`

6.3 Parâmetros Principais do SLURM

Parâmetro	Descrição
<code>-job-name</code>	Nome exibido na fila
<code>-ntasks</code>	Número total de processos do job no <i>cluster</i> inteiro
<code>-ntasks-per-node</code>	Número de processos executados em cada nó alocado
<code>-partition</code>	Fila de execução: <code>cpu</code> ou <code>gpu</code>
<code>-gres=gpu:N</code>	Solicita <i>N</i> GPUs para o job (apenas para <code>-partition=gpu</code>)
<code>-cpus-per-task</code>	Número de núcleos de CPU por processo
<code>-mem</code>	Memória RAM total (ex.: <code>8G</code> , <code>32G</code>)
<code>-nodelist</code>	Aloca o job para o nó especificado
<code>-time</code>	Limite de tempo no formato <code>DD-HH:MM:SS</code>
<code>-output</code>	Arquivo para a saída padrão (<code>%j</code> = job ID)
<code>-error</code>	Arquivo para a saída de erros
<code>-mail-type</code>	Eventos que disparam e-mail: <code>BEGIN</code> , <code>END</code> , <code>FAIL</code>
<code>-mail-user</code>	Endereço de e-mail para notificações

É relevante ressaltar que para `-partition=cpu`, o parâmetro `-gres=gpu:N` deve ser removido do *script* SLURM.

Quantidade de Processos e Paralelismo

Para o contexto do *cluster* Heisenberg, é necessário que o job sempre seja submetido com base em um *script* SLURM que contenha os parâmetros `-ntasks=1` e `-ntasks-per-node=1`, a fim de evitar a alocação de recursos para processamento paralelizado, priorizando uma filosofia de maior disponibilidade de recursos ao maior número de usuários possível.

7 Submissão, Monitoramento e Diagnóstico

7.1 Submetendo e Cancelando Jobs

```
sbatch script.sh          # submete o job para a fila

squeue                    # lista todos os jobs
squeue -u $USER           # filtra pelos jobs do usuario
```

```
watch -n 5 squeue -u $USER      # atualiza automaticamente a cada
                                5 segundos

scancel <JOB_ID>                 # cancela um job específico
scancel -u $USER                 # cancela todos os jobs do
                                usuario
```

7.2 Monitorando a Execução

```
# Acompanhar a saída em tempo real
tail -f logs/saida_<JOB_ID>.out

# Ver informacoes detalhadas do job
scontrol show job <JOB_ID>

# Ver uso de GPU (atualiza a cada segundo)
watch -n 1 nvidia-smi

# Ver uso de CPU e memoria
htop
```

8 Workflow Completo

O *workflow* completo desde o acesso ao HPC Heisenberg e a conclusão de um job envolve o seguinte conjunto de passos:

1. Acesso ao cluster: `ssh usuarioXXX@heisenberg.cnpem.br`
2. Redirecionamento para o diretório desejado (comandos bash de navegação e inspeção)
3. Submissão do job ao *scheduler* SLURM: `sbatch arquivo_job.sh`
4. Para Jupyter Notebook:
 - Executa-se: `cat slurm-<jobid>.out`
 - Acesso ao link: `http://172.20.10.15:8888/...`
 - Finalizado o uso, cancelar o job: `scancel <jobid>`
5. Para *script* Python:
 - Monitoramento via logs

9 Boas Práticas

9.1 Uso Responsável do *cluster*

- **Cancele jobs** que não são mais necessários: `scancel <JOB_ID>`
- **Nunca execute computações pesadas no login node** — isso impacta todos os usuários conectados
- **Respeite as cotas de disco**: o sistema de arquivos compartilhado tem capacidade finita
- **Comunique falhas de hardware** à equipe responsável
- **Não compartilhe sua senha** de acesso com outros usuários

9.2 Organização de Experimentos

Manter uma estrutura de diretórios consistente facilita a reprodução de experimentos e a colaboração:

```
projeto/  
+-- data/           # dados brutos e/ou pre-processados  
+-- src/            # código-fonte Python  
|   +-- train.py  
|   +-- evaluate.py  
+-- configs/        # arquivos de configuração (YAML, JSON)  
|   +-- exp_01.yaml  
+-- logs/           # saídas do SLURM (.out e .err)  
|   +-- saida_123.out  
|   +-- erro_123.err  
+-- results/        # métricas, gráficos e checkpoints  
+-- environment.yml  # ambiente conda para reprodução  
+-- job.sh           # script SLURM de submissão  
+-- README.md        # documentação do projeto
```

9.3 Boas Práticas de Submissão

1. **Crie o diretório de logs antes de submeter**: `mkdir -p logs`. Se o diretório não existir, o SLURM não conseguirá criar os arquivos de saída e o job falhará silenciosamente.
2. **Teste localmente com dados pequenos** antes de submeter para o *cluster*. Um script que falha por erro de sintaxe não deve ocupar recursos do *cluster*.
3. **Estime os recursos de forma realista**. Superalocar memória bloqueia outros usuários; subalocação causa falhas por OOM (*Out of Memory*).

4. Use `-time` para evitar que jobs travados consumam recursos indefinidamente.
5. **Organize as execuções:** mantenha um diretório por experimento, com script SLURM, logs e resultados juntos.

9.4 Logging e Registro de Execução

Todo script executado no *cluster* deve possuir algum mecanismo de logging estruturado. Em ambientes HPC, onde a execução geralmente ocorre de forma assíncrona e sem interação direta, o log é a principal fonte de diagnóstico e rastreabilidade.

```
1 import logging
2
3 logging.basicConfig(
4     filename='logs/run.log',
5     level=logging.INFO,
6     format='%(asctime)s | %(levelname)s | %(message)s'
7 )
8
9 logging.info("Inicio da execucao do job")
10
11 # Exemplo de etapas genericas
12 logging.info("Carregando dados de entrada")
13 logging.info("Processando dados")
14 logging.info("Salvando resultados")
15
16 logging.info("Execucao finalizada com sucesso")
```

Boas práticas de *logging* em HPC:

- Registrar data e hora de início e fim da execução
- Registrar parâmetros de entrada do *script* (argumentos, arquivos, configurações)
- Registrar etapas principais do processamento
- Registrar informações sobre uso de recursos (tempo, memória, etc., quando possível)
- Registrar erros e exceções de forma clara
- Evitar uso excessivo de `print` em favor de logs estruturados

Exemplo com tratamento de erros:

```
1 import logging
2
3 logging.basicConfig(
4     filename='logs/run.log',
5     level=logging.INFO,
```

```
6     format='%asctime)s | %(levelname)s | %(message)s'
7 )
8
9 try:
10     logging.info("Iniciando execucao")
11
12     # Simulacao de tarefa
13     resultado = 10 / 0
14
15     logging.info("Execucao concluida")
16
17 except Exception as e:
18     logging.error(f"Erro durante execucao: {e}", exc_info=
        True)
```

Por que isso é importante em HPC?

- Jobs podem executar por horas ou dias sem supervisão
- Não há acesso interativo durante a execução
- Logs permitem reproduzir, auditar e depurar experimentos
- Integracao com sistemas de agendamento (ex: SLURM) facilita o rastreamento

9.5 Reprodutibilidade

Sendo a reprodutibilidade um dos pilares das boas práticas científicas modernas, no contexto de experimentos computacionais, ela exige:

- **Seeds fixas:** fixar a semente aleatória em todas as bibliotecas utilizadas.

```
1 import random, numpy as np, torch
2 random.seed(42)
3 np.random.seed(42)
4 torch.manual_seed(42)
```

- **Versionamento do ambiente:** manter `environment.yml` ou `requirements.txt` no repositório.
- **Versionamento do código:** usar `git` e registrar o *commit hash* nos logs de cada execução.
- **Parametrizar tudo:** evitar valores *hardcoded* no código. Use arquivos de configuração (`.yaml`) ou `argparse` para todos os hiperparâmetros.

10 Erros Comuns e Debugging

10.1 Job Preso em PENDING

Causa: os recursos solicitados (CPUs, GPUs, memória ou tempo máximo) não estão disponíveis na partição atual.

Diagnóstico:

```
squeue -u $USER -l      # observe o campo REASON
sinfo                   # veja o estado de cada nó e particao
```

Soluções:

- Reduzir `-mem` ou `-time`
- Mudar de `-partition`
- Aguardar liberação de recursos (jobs de outros usuários terminarem)

10.2 Falta de Memória (*Out of Memory*)

OOM Error

O job tentou alocar mais memória do que o declarado no script SLURM. O sistema encerra o processo automaticamente, e o job aparece com status OOM ou FAILED.

Diagnóstico:

```
sacct -j <JOB_ID> --format=JobID,MaxRSS,ExitCode
```

Soluções:

- Aumentar `-mem` no script SLURM
- Reduzir o *batch size* na linha de comando Python
- Processar dados em *chunks* menores

10.3 Uso Incorreto de GPU

- **GPU não alocada, mas o código tenta usá-la:** erro CUDA device not available. Verifique se `-gres=gpu:1` está no script SLURM e se a partição correta foi especificada.
- **Dados na CPU, modelo na GPU:** erro de dispositivo no PyTorch. Garanta que tensores e modelos estejam no mesmo dispositivo com `tensor.to(device)`.
- **GPU ociosa durante job:** o código não está usando CUDA. Monitore com `nvidia-smi` e verifique se o modelo foi enviado para o dispositivo GPU com `model.to(device)`.

10.4 Kernel Morto no Jupyter / Dependências Quebradas

- **Morte do kernel:** geralmente causada por consumo excessivo de RAM. Prefira scripts para jobs longos.
- **Dependências quebradas:**

```
# Remover e recriar o ambiente do zero
conda env remove -n meu_env
conda env create -f environment.yml
```

- **Módulo não encontrado no job:** o ambiente conda não foi ativado no script SLURM. Adicione `source ~/.bashrc` e `conda activate nome_env` no início do script.

A Tutorial de Acesso à VPN

Para acessar a VPN da rede interna do CNPEM, siga as etapas:

1. Pressione a tecla **Windows**, busque pelo aplicativo nomeado FortiClient e abra-o
2. Faça seu login, informando usuário, senha (a mesma do computador) e efetuando a verificação em duas etapas
3. Na opção "**Nome da VPN**", selecione CNPEM-SSL
4. Clique em "**Conectar**"
5. Aguarde o estabelecimento da conexão com a rede do CNPEM

B Referência Rápida de Comandos SLURM

Ação	Comando
Submeter job	<code>sbatch job.sh</code>
Ver fila (todos)	<code>squeue</code>
Ver fila (meu usuário)	<code>squeue -u \$USER</code>
Cancelar job	<code>scancel <JOB_ID></code>
Cancelar todos os meus jobs	<code>scancel -u \$USER</code>
Detalhes de um job	<code>scontrol show job <JOB_ID></code>
Histórico de jobs	<code>sacct -u \$USER</code>
Estado dos nós	<code>sinfo</code>

Cheat Sheet de comandos SLURM: <https://slurm.schedmd.com/pdfs/summary.pdf>

C Referência Rápida de Comandos Conda

Ação	Comando
Listar ambientes	<code>conda env list</code>
Criar ambiente	<code>conda create -n nome python=3.13</code>
Criar a partir de YAML	<code>conda env create -f environment.yml</code>
Ativar ambiente	<code>conda activate nome</code>
Desativar ambiente	<code>conda deactivate</code>
Exportar ambiente	<code>conda env export > environment.yml</code>
Remover ambiente	<code>conda env remove -n nome</code>
Listar pacotes	<code>conda list</code>